

Design Report Week 1

Tim Solfronk, Bhargav Joshi and Johan Roos

Source code: <https://github.com/johanhroos/ID2221-labs>

Task 1: Our Data Catalog

Taxi Trips

Each parquet record represents a reported taxi trip, which is not necessarily unique. We found negative **total_amount** entries that match other trips' **total_amount** having a positive amount. These likely represent reimbursed trips.

When using a candidate primary key combination consisting of the attributes **vendor_id**, **tpep_pickup_datetime**, **tpep_dropoff_datetime**, **pulocation_id**, **dolocation_id**, and **trip_distance** we find that the maximum amount of duplicate rows is 2 having opposite signed **total_amount**. When including **total_amount** in the composite key, we only get unique entries. As there is no natural primary key, we combine the seven entries as a candidate primary key. It is highly unlikely for a trip to take place between a pickup zone and a dropoff zone such that the total cost is the same and start/end times are the same because the timestamps are accurate to the second.

Location IDs join to the zone lookup dataset and timestamps join to environmental observations. Both trip timestamps are temporal attributes. Vendor, rate code, payment type, location IDs, and the store-and-forward flag are categorical. New trips increase row counts and extend the timestamp range.

Weather

Each CSV record represents an hourly observation. The primary key, temporal attribute, and join field is **timestamp**, which is constructed from **year**, **month**, **day** and **hour**.

Measurements include temperature, humidity, precipitation, wind, and pressure. **Coco** is categorical and is represented by integers. The dataset grows with observation hours.

Air quality

Each csv record represents a sensor measurement. The primary key combines **state code**, **county code**, **site num**, **parameter code**, **POC** and **timestamp local**. State/county codes and timestamps truncated to an hour support joins. Temporal attributes are **timestamp local**, **timestamp gmt** and **date of last change**. Parameter, method, qualifier, unit, geographic codes, and datum are categorical. Geographic codes remain strings to preserve leading zeros. Rows grow with observation hours, sensors, and parameters

Taxi Zone Lookup

Each CSV record describes a zone. **Location ID** is the primary key and joins to pickup/dropoff location IDs. Zone, borough, service zone, and derived state/county codes are categorical. There is no temporal field or validity period.

Task 2: Our Storage Architecture

Directory structure

```
ID2221_Lab/
├── data/                # Raw source datasets; original formats retained
├── configs/
└── └── datasets/        # Schemas, keys, validation, transformation rules
```

delta_tables/	
cleaned/	# Standardized and validated Delta tables
integrated/	
integrated_taxi_trips/	# Enriched taxi-trip Delta table
src/	
config.py	# Paths, dataset registry, integration configuration
spark.py	# Spark and Delta session configuration
run.py	# Full pipeline entry point
datasets/	# Dataset-specific transformations
taxi.py	
weather.py	
air_quality.py	
zones.py	
pipelines/	# Main pipeline stages
ingest.py	
integrate.py	
benchmark.py	
utils/	# Generic reusable components
ingestion.py	
integration.py	
keys.py	
ingestion_logger.py	
Dockerfile	# Reproducible Spark/Delta environment
ReadMe.md	# Setup and execution instructions

Table and column names use lowercase snake case. Taxi zone dataset is a conventional lookup table because it contains one small, relatively stable record per **location_id**. It is used to translate pickup and dropoff location IDs into zone and borough names. Weather and air-quality data are observation tables rather than traditional lookup tables.

The current production Delta tables are unpartitioned. This is appropriate for Taxi Zones because it contains only 265 rows, and for weather because the supplied dataset contains only 8,784 hourly observations. Partitioning these small tables would create many small files and add unnecessary metadata and scheduling overhead.

For workloads that commonly filter by a date range, taxi trips and integrated trips should be partitioned by **pickup year** and **pickup month**. This allows Spark to skip unrelated month partitions. The same applies to air quality and weather at larger scales. Partitioning in general becomes harmful when partitions are too small or data is unevenly distributed.

Task 3+4: The Ingestion Pipeline

Before going into how the ingestion pipeline works exactly, we should first define exactly what the pipeline should do. The obvious part is loading the dataset and saving it as a delta table. It should also follow the directory structure and naming conventions proposed in Task 2. We might want to map some columns to different values to make it easier to join them with other tables. This includes normalizing timestamps, but also mapping columns like “borough -> county code”. It should also handle null values in columns by replacing them or removing

the respective row when preferred. And lastly, it should make sure the data that we ingest is of some standard quality by removing duplicate and invalid rows.

Most of the tasks mentioned above can actually be implemented very generically, given you provide a config that includes data-specific requirements. We designed the following config format for this:

A config file is a .json file with the attributes...

- **"table_name"**: the name of the delta table that should be created
- **"primary_key"**: a list of column names that should make up the key of the dataset, this can include columns that don't exist yet (columns that are created during ingestion)
- **"columns"**: a list of column elements containing the following info
 - **"name"**: the name we want the column to have, this does not have to be the name it has right now (in the csv-header or .parquet schema)
 - **"type"**: the type of data that is in this column, includes most standard times, but could be improved later on
 - **"nullable"**: a boolean, whether the row is allowed to be null
 - **"default"**: an optional replacement value for nulls in a non-nullable column. If nullable is false and no default is specified, rows with a null value in that column are removed. If nullable is true, null values are retained.
 - **"keep_column"**: a boolean that states if the column should be kept in the final delta table or if it can be deleted
- **"extra_functions"**: a list of named dataset-specific transformations. The dispatcher currently supports:
 - ["add_county_from_borough", "separate_time_to_timestamp", "dates_and_times_to_timestamp", "add_negative_fare_flag", "add_likely_reimbursement_flag"]

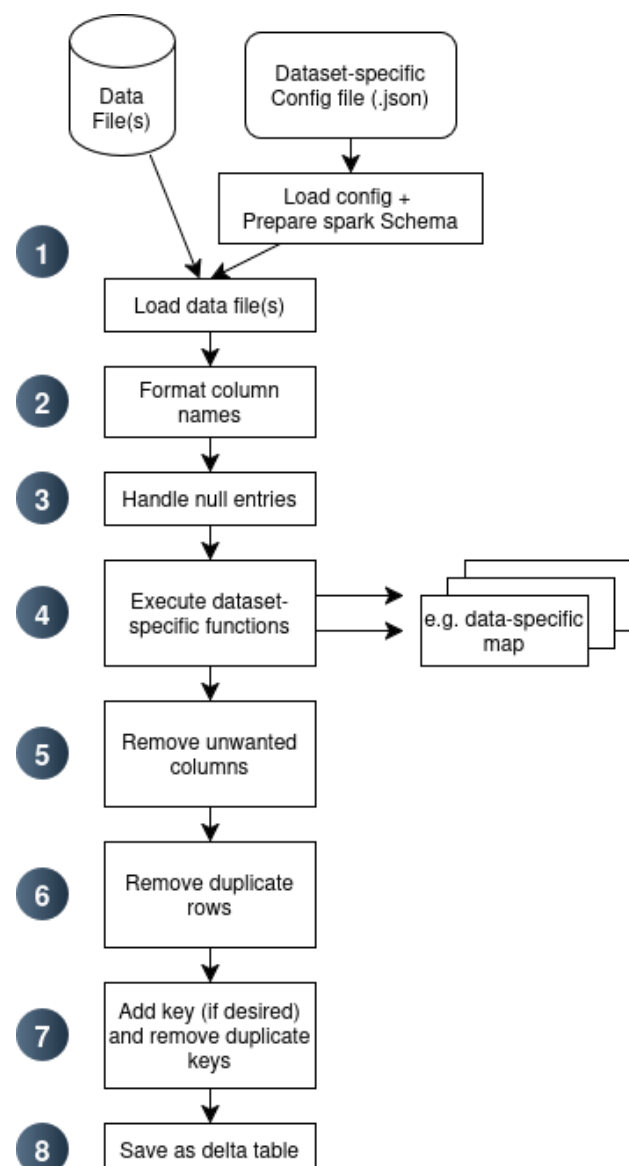
Using this, we build the following pipeline:

1. Loading the config and data file(s):

- a. This loads the config file and creates a schema from it
- b. It loads the data files according to the configured input format. It then validates the source columns against the configuration and casts values to the configured data types
- c. The pipeline validates that the source schema matches the configured columns, removes rows whose values cannot be cast to the required types, and then standardizes the configured column names.

2. Format column names:

- a. This converts configured output names to lowercase snake case and verifies



that the resulting names are valid and unique

3. Handle null entries:

- a. This goes through all columns and if nullable=false in the config:
 - i. Either delete the respective row or replace it with “default” from the config

4. Execute dataset-specific functions:

- a. This is the only part that is not generic
- b. Example functions are:
 - i. Converting time columns that are not “timestamp_ntz”
 - ii. Adding new attributes by mapping the data to help joins

5. Remove unwanted columns:

- a. If some columns have uninteresting data for us, we can delete them here if keep_column=false

6. Remove duplicate rows

7. Key Management:

- a. Add a new key column if desired
- b. Validate that no primary-key value is missing and that no duplicate primary-key combinations remain. Exact duplicate rows are removed earlier. Remaining duplicate keys cause ingestion to fail rather than silently discarding records

8. Save as delta-table

The checking of the input arguments and error handling of the code is omitted in the illustration.

As outlined above, most components are generic and reusable for any data. Even the dataset-specific functions are implemented in a reusable way. (e.g. they still work, if some columns are missing)

This way, even if we added 20 new datasets, other than writing a small config for all of them, not a lot of adjustments would have to be made to the ingestion pipeline.

Lastly, we also added a logger that writes all important changes and transformations in a log file. This also includes reporting the timeline and how many rows were deleted during each step.

Our Data standards:

- All our timestamps are represented in the format “timestamp_ntz”
- We have also laid out all possible datatypes in the inside the ingestion pipeline and enforce the format of names / rename columns, when it is needed

Task 5: The Integration Pipeline

The integrated Delta table preserves every cleaned taxi record and adds pickup/dropoff zone and borough, hourly weather, and an air-quality measurement with its parameter name and unit.

We join the zone lookup using **pulocation_id** and **dolocation_id**. Pickup state/county codes are derived from the five New York City boroughs and EWR. Unmapped boroughs receive null values.

Weather is joined on the pickup hour e.g. a 10:37 pickup receives the 10:00 observation. Each observation represents the hour beginning at its timestamp. The supplied weather series applies to all zones.

For air quality, we retain state/county combinations present in the zone lookup, convert **timestamp_gmt** from UTC to **America/New_York**, and average measurements by state, county, and hour. Joining this summary to pickup geography and hour prevents multiple sensors from multiplying taxi rows. Air quality measurements are average for that hour.

Left joins preserve trips with missing context, leaving contextual values null without interpolation or imputation. Before writing, the pipeline verifies that the row count is unchanged.

Hourly matching approximates pickup conditions, while county averages weight measurements equally and hide spatial differences. Taxi and weather timestamps are assumed to represent New York local time. UTC conversion handles daylight-saving offsets, but timezone-free taxi timestamps cannot distinguish repeated local hours, which may combine observations.

Task 6: Benchmarking

We compared unpartitioned and pickup-year/month Delta layouts using our shared ingestion function and Delta writer. Both stored identical cleaned records and columns, including generated identifiers, quality flags, and partition columns.

Ingestion timing covered loading, validation, transformations, deduplication, key generation/checks, repartitioning, and a fresh Delta write. It excluded Spark startup, separate profiling, ingestion logging/metadata, verification, and integration. Each layout had three write trials with alternating order, eight output partitions, and a 100,000-record file limit. Both experiments used x86-64 Linux, Spark 3.5.6, Delta 3.2.0, two local Spark threads, a 2 GiB driver heap, and eight shuffle partitions.

We measured ingestion time, storage size, file count, and the three required queries: trips per pickup borough, average duration in seconds per pickup day, and average **fare_amount** per pickup borough. Missing boroughs form an Unknown group. Duration excludes null/negative values; fare excludes null/non-finite values and retains finite negative fares.

Each query/layout had one warm-up and three timed runs on the final written table. Timing included reading and collecting results. Spark's data cache was cleared between runs; filesystem and JVM caches remained warm. All writes passed row-count checks, and query results matched across layouts and repetitions within floating-point tolerance.

Results

January 2024 contained 2,964,624 source rows and 2,964,568 cleaned rows; January–March contained 9,554,778 and 9,554,597 respectively. Cleaning and deduplication removed 56 and 181 rows. The table reports medians over three trials.

Input	Layout	Ingestion median (s)	Active data (MiB)	Data files	Trips/bo rough median (s)	Duration/day median (s)	Fare/bor ough median (s)
January	Unpartitioned	47.674	259.79	32	0.249	0.242	0.256
January	Pickup year/month	47.520	259.78	46	0.231	0.235	0.234
January–March	Unpartitioned	138.526	843.22	96	0.338	0.440	0.362
January–March	Pickup year/month	139.295	838.82	118	0.335	0.420	0.407

Storage figures cover active Delta data. January–March ingestion ranged from 136.985–138.736 seconds unpartitioned and 139.234–139.907 seconds partitioned. Complete timing ranges and total directory metrics, including logs and filesystem metadata, accompany the raw results.

Limitations

A 3.22× increase in cleaned rows increased ingestion time by 2.91× unpartitioned and 2.93× partitioned. Storage grew by 3.25× and 3.23× respectively: approximately proportional growth over the measured inputs.

Monthly partitioning used 0.5% less storage for January–March but produced 22 more files and took 0.6% longer to ingest. Query medians favored different layouts, with overlapping ranges. We retain unpartitioned storage for its simpler layout and fewer files.

All required queries scan every month, so they do not benefit from partition pruning. Date-filtered queries could yield different results. Sparse pickup dates can also create small partitions.

Ingestion processes and writes full records; queries read selected columns and return small aggregates. Validation and writing may repeat transformation work; caching the cleaned result is an unmeasured optimization.

Warm caches, local resource contention, two input sizes, and three repetitions limit generalization to cold-disk, distributed, or 20× workloads. Complete January and

January–March summaries link the findings to measured runs. Each directory retains CSV timings and `results.json` with query outputs and environment metadata.